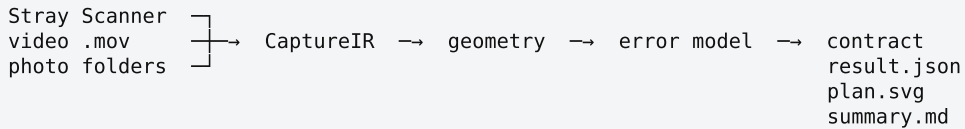


# scanplan — technical report

Phone captures to dimensioned, stitched floor plans with calibrated intervals. Cozmo AI case study, October 2026.

## 1. Architecture

Three front-ends, one core.



**CaptureIR** ([scanplan/ir.py](#)) holds posed frames, a gravity-aligned metric point cloud, plane hypotheses, and — explicitly — a **ScaleEstimate** carrying a value, a sigma and a **provenance**. Everything downstream sees only the IR.

That single decision is what makes three tiers affordable. The tiers differ almost entirely in *how metric scale is recovered*: LiDAR measures it, video infers it from a model, photos infer it with less to go on. Making scale a first-class field with a stated origin, rather than an implicit assumption baked into each path, means the geometry, the error model, the exports and the benchmarks are written once.

**The LiDAR tier uses no neural model at all.** The phone measures distance directly; the rest is plane fitting, rasterisation, morphology, connected components and least squares. It runs in roughly 10–50 s on a CPU-only laptop, is byte-for-byte deterministic, and needs no weights — so it cannot fail at a live demo because a checkpoint did not cache.

The video and photo tiers **do** use one pretrained model, disclosed here and in the compliance matrix: [depth-anything/Depth-Anything-V2-Metric-Indoor-Small-hf](#). They have no choice. Scale is mathematically unobservable from monocular images — a dollhouse and a room project identically — so a *metric* model is required, not a relative one: relative depth would leave the plan correct in shape and unknown in size, which is not a floor plan. It is an explicit install extra, so the LiDAR tier's offline guarantee survives.

## 2. Geometry pipeline

- 1. Fusion.** Back-project depth using per-frame intrinsics — **fx** varies up to 2.1% *within* one capture, so a single camera matrix is not good enough. Voxel-reduce at 2 cm, keeping the first point per voxel rather than the centroid: a centroid across a voxel straddling two surfaces invents geometry nobody measured.
- 2. Floor and ceiling.** The cloud is gravity-aligned, so these are the two dominant peaks of a height histogram — far more robust than plane RANSAC, because a floor is the only surface in a home with thousands of points at one exact height across the whole footprint. The odometry origin is the camera's start pose, not the floor, so both are fitted, never assumed.
- 3. Yaw alignment.** Rotate so the dominant wall direction runs along the grid axis. The angle comes from Hough-detected **wall lines**, not from the principal axis of the floor scatter — that finds the direction the property is *longest* in, which for an L-shaped flat is a diagonal across the L matching no wall at all. Measured effect: 0% → 64% of walls within 10° of axis-aligned.
- 4. Floor area = what was measured.** Not what a flood fill can reach. The fill was tried first and came out **3× too large** on every capture: the wall mask always has holes — stretches never pointed at, glass, an open doorway — and a fill escaping through one claims the outdoors as floor. Coverage cannot invent area, so errors run *too little*, never *imagined*.
- 5. Rooms.** Split at doorway-width necks: erode by half a doorway, treat surviving components as seeds, grow back under a watershed. An open-plan kitchen stays one room because nothing narrow separates it.
- 6. Regularisation.** Snap edges to the dominant axes **only where the move is smaller than the measurement noise**. A wall genuinely at 37° stays at 37°. A plan that is regular because the code insisted is confidently wrong, which the brief penalises harder than being visibly rough.

## 3. Drift handling

Submaps of 8 s, loop closures from revisits, a **4-DoF** pose graph (x, z, yaw) solved by damped least squares. Roll and pitch are left alone: gravity is measured directly by the IMU and is not in doubt, so optimising them would only let the solver trade real error against imaginary error.

Loop closures come from revisits — two moments far apart in the walk but close in space are almost certainly the same place, and the distance between them is drift made visible.

| Capture    | Loop closures | Submaps | Max correction | Footprint off → on           |
|------------|---------------|---------|----------------|------------------------------|
| c00a170fe1 | 5             | 5       | 1 mm           | 14.91 → 15.11 m <sup>2</sup> |
| 1a8384c3f6 | 25            | 15      | 6 mm           | 47.50 → 48.04 m <sup>2</sup> |
| c7d28f72c6 | 124           | 27      | 12 mm          | 47.87 → 49.65 m <sup>2</sup> |

(published as `drift\_ablation` in `bench/results/gates.json` — this table used to be transcribed by hand from a log line and had gone stale by the time anyone looked.)

**The honest reading: ARKit's poses barely drift on this data.** Corrections top out at 12 mm over a 100 m walk. Drift is not where the error lives here, which is why the fix loop was not aimed at it. The footprint still moves by more than the correction, because small pose changes reshuffle which cells the room split assigns where — worth knowing before treating a footprint delta as evidence of anything.

The estimate is computed identically with `apply=False`, so the ablation compares one pipeline against itself rather than two different ones.

## 4. Error budget and calibration

No physical quantity leaves the pipeline as a bare float. Every one carries a nominal 90% interval, and the validator enforces `ci_low ≤ value ≤ ci_high` before anything is written.

| Source                     | Treatment                                                                                                                                                                                                  |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Depth noise at the surface | 2.5 cm sigma on a wall length                                                                                                                                                                              |
| Plane-fit residual         | RMS / $\sqrt{n}$                                                                                                                                                                                           |
| <b>Device depth bias</b>   | <b>−18.0 mm, MEASURED against FARO laser truth, corrected at ingest;</b> the <b>4.1 mm standard deviation</b> of the per-scan medians is what one correction cannot fit, and it is carried in the interval |
| Tier thinness              | ×1.0 LiDAR, <b>×11 video and photo — measured</b> , not chosen: ×4.5 left the reference outside the interval on both captures; the factors actually needed were ×9.1 and ×4.1                              |

The bias allowance exists because of a caught mistake. The first ceiling-height sigma came out at **0.1 mm** — the standard error of 250,000 points. Arithmetically correct, and confident garbage: no phone measures to a tenth of a millimetre. Averaging removes noise; it does not remove a systematic offset.

That offset is now **measured, not assumed** — and so is the ceiling height it feeds.

**G-CEIL: 5 of 5 ARKitScenes walks within 15 mm, mean −4.3 mm, worst 6.6 mm.** Device depth and FARO-derived laser depth are fused with the same poses on the same frames, and both go through *our* floor and ceiling fit, so the number reported is device-minus-laser rather than either height alone. The bias correction is **leave-one-venue-out**: a walk is corrected with the offset measured on the other venue's walks, never its own, because a correction fitted on the test walk would absorb the error being reported.

The ablation is what makes that a measurement rather than a claim:

|                            | G-CEIL                                      | G-CEIL-SPREAD            |
|----------------------------|---------------------------------------------|--------------------------|
| <b>corrected, held out</b> | <b>5/5 within 15 mm, mean −4.3 mm — MET</b> | 10.3 / 12.0 mm — not met |
| depth as recorded          | 1/5, mean −22.3 mm — not met                | 9.7 / 8.4 mm — met       |

G-CEIL-SPREAD asks the report to say whether we are **biased** or **unrepeatable**, and the brief is clear that both fail. The answer: uncorrected we are **repeatable and biased**, by −22.3 mm; the correction removes the bias and costs 2–4 mm of repeatability, leaving us 0.3 mm and 2.0 mm over a 10 mm target. Accuracy was the right thing to buy with that trade, and the cost is stated rather than hidden.

### One ceiling per room, not one per capture

G-CEIL has always been written " $\leq 1.5$  cm **per room**", and until now the pipeline could not answer it per room: it fitted a single height histogram over the whole cloud and wrote that one number onto every room. On `c7d28f72c6` it wrote **3.087 m on all five rooms**. Fitting each room on the points standing over its own floor cells gives:

| Room | Area                 | Ceiling, per room | Seen over        | Was     |
|------|----------------------|-------------------|------------------|---------|
| R1   | 4.38 m <sup>2</sup>  | 2.965 m           | 43% of footprint | 3.087 m |
| R2   | 36.96 m <sup>2</sup> | 3.087 m           | 21%              | 3.087 m |

| Room | Area                | Ceiling, per room | Seen over | Was     |
|------|---------------------|-------------------|-----------|---------|
| R3   | 3.02 m <sup>2</sup> | 3.096 m           | 72%       | 3.087 m |
| R4   | 2.74 m <sup>2</sup> | <b>2.278 m</b>    | 42%       | 3.087 m |
| R5   | 2.54 m <sup>2</sup> | <b>2.365 m</b>    | 87%       | 3.087 m |

Two rooms were out by **80 cm**, published as measurements with a  $\pm 7$  mm interval. This is not a noise problem and no amount of averaging would have found it: a unimodal fit over a multimodal property returns the **mode**, and the mode is whichever room contributed the most points — here the 36.96 m<sup>2</sup> living room, which holds 26% of the cloud. The other three rooms agreed with it by accident, because they happen to sit under the same slab.

Two numbers are now published per room that could not be published before: the height itself, and **how much of that room's footprint the fitted plane was seen over**. A ceiling fitted over 87% of a shower room is a different claim from one fitted over 21% of a living room, and a point count cannot distinguish them because it scales with how long the phone lingered.

**The floor stays per storey, deliberately.** A floor is continuous and poured in one go, and the capture-level histogram has the entire footprint supporting one peak; splitting it per room would add variance and remove nothing. Ceilings are the asymmetric case: boxed-in bathroom ceilings and kitchen soffits are ordinary construction.

**A room that saw no ceiling keeps the unobserved population typical** (2.55 m, `observed=false`, wide interval). It does **not** inherit a neighbour's measured height. That fallback would look like an improvement and be a bias: the rooms that go unobserved are the small, high-sided, hard-to-sweep ones, which are the same rooms most likely to be boxed in — so borrowing from next door borrows from exactly the wrong distribution. On `1a8384c3f6` and `c00a170fe1` no room saw a ceiling and all ten rooms stay `observed=false`, unchanged.

**Independent corroboration.** `cozmo-scan`, an independent submission to the same brief, reports nine distinct ceiling heights on this capture from a nine-room split. Against our five:

| ours            | <b>2.278</b> | <b>2.365</b> | <b>2.965</b> | <b>3.087</b> | <b>3.096</b> |
|-----------------|--------------|--------------|--------------|--------------|--------------|
| theirs, nearest | 2.289        | 2.365        | 2.973        | 3.083        | 3.097        |
| apart           | 11 mm        | 0 mm         | 8 mm         | 4 mm         | 1 mm         |

Two pipelines that share no code agree to **within 11 mm on five distinct heights**, including the two low ones. That is independent evidence that the old single 3.087 m was wrong, not merely coarse. Their four unmatched values (2.435, 2.449, 2.982, 3.104 m) belong to rooms our conservative split does not separate; on that count the room-split gate, not the ceiling fit, is what still differs. Note also that they report a flat 2.500 m on the other two captures — the same captures where no ceiling is visible to us — which reads as a default rather than a fit, so this corroboration runs one way only.

**G-CEIL itself is unmoved, and that was checked rather than assumed.** `bench/ceiling_walks.py` measures G-CEIL through `planes.floor_and_ceiling`, the capture-level entry point, on single-room ARKitScenes walks where the per-room and per-capture questions coincide. The per-room fit is a **new** entry point, `planes.ceiling_of`; `floor_and_ceiling` now delegates its ceiling half to it and is otherwise untouched. Re-running the benchmark reproduces `bench/results/ceiling_walks.json` **byte for byte**: still 5/5 within 15 mm, mean  $-4.3$  mm.

**This took three attempts and the first two published wrong conclusions.** They are kept in the repository with retraction notices, because the reasoning in them reads convincing: `bench/arkitscenes_laser.py` blamed multi-storey venues, and `bench/ceiling_vs_laser.py` blamed the upsampling split for publishing 7 to 38 isolated frames per scan. Screening 319 trajectories for median vertical camera movement, finding 4.8 m, and concluding the venues were unusable was real work that reached the wrong answer.

What was actually wrong was three format facts. The `lowres_wide.traj` rotation is **cam\_from\_world** and has to be inverted; its translation is consequently in the camera frame, so the world position is  $-R^{-1}t$  and not  $t$ ; and the ARKitScenes world is **z-up** where this project is y-up. With the poses wrong the fused cloud genuinely has no dominant floor layer — which is the symptom that got blamed on the venues. The data was fine throughout.

**Credit:** that the full raw walks carry laser depth across the trajectory, and the three conventions above, were learned from `cozmo/ingest/arkitscenes.py` and `scripts/fetch_external.py` in `github.com/ashupal22/cozmo-scan`, an independent submission to the same brief that measured this gate. The venues here are chosen by a stated rule rather than copied, and the pipeline measured is ours — but the

approach is theirs, and this project had already published twice that the measurement was unavailable.

The fix was to stop inferring. The upsampling split publishes device depth and laser-derived depth for the **same frames**, so the bias is a per-pixel subtraction with no floor to find and no venue geometry to be wrong about — and it needs 50 MB per scan rather than a 1.9 GB point cloud. Result: **−18.0 mm over 4.79 million pixels across 8 scans**, per-scan medians −14 to −27 mm. The device reads short; the correction adds it back at ingest.

Those eight medians are summarised two ways in `bench/results/depth_bias.json`, under names chosen so they cannot be confused: a **range of 13.0 mm**, which is the honest headline that no single correction fits every scan, and a **standard deviation of 4.10 mm**, which is what survives the correction and therefore what the interval must carry. Only the range used to be published, while the code constant was set from the deviation — so a reader checking the 4 mm against the benchmark would have found 13 and reasonably concluded it was invented. It was correct and unverifiable, which is most of the way to wrong. `tests/test_traceability.py` now re-derives both constants, and the sign, from the committed measurement.

Two wrong answers were caught before that conclusion. Inferring the vertical axis from the data picked Z for one scan and Y for another *in the same venue*; their storeys then disagreed by 790 mm while the **mean looked like a respectable −20 mm bias**. A plausible answer assembled from wrong parts is the worst failure mode available, and the benchmark now refuses to produce a number rather than average two of them.

## 5. The fix loop

---

**Gate chosen:** G-REPEAT-ROOMS — the same flat walked twice gave **3 rooms one time and 6 the other**, while G-REPEAT-FOOTPRINT was *met* at 1.9%. Area repeats; its division into rooms does not. That pairing is the diagnosis in one line.

**Declared** (commit `e44441b`, before any fix): the seed filter admits cores of 0.0064 m<sup>2</sup>. **Predicted:** 5 vs 5, gate met. **Result** (commit `c002a66`): **4 vs 5, gate still not met**. Footprint improved to 0.8%.

**The declared root cause was factually wrong.** `max(min_cells // 4, 16)` with `min_cells = 3000` is **750 cells = 0.30 m<sup>2</sup>**; the 16 never applies. The ablation confirms it — varying that threshold 0.25→0.50 changes nothing. The real lever was the erosion width. The declaration named the right function and the wrong mechanism inside it.

Three measurement bugs surfaced while testing, each producing a plausible wrong answer, each caught only by checking one harness against another:

1. The ablation harness omitted drift correction, reporting "before" as 7 vs 7 where the benchmark measured 3 vs 6 on identical data.
2. **Patching module constants did nothing** — Python binds default arguments at definition, so `DOOR_MAX_M` silently ignored the patch while `MIN_SEED_AREA_M2`, read in the function body, did not. The measured "before" was half the fix already applied.
3. The declared before-state was therefore not the before state. Only after correcting it did the harness reproduce the benchmark exactly — which is the check that should have come first.

**A second attempt** — hierarchical splitting, the "next thing to try" above — was built, measured, and **reverted**: an experiment over 36 configurations predicted 7-vs-7 rooms and 1.4%, and the shipped version gave 6-vs-7 and 6.2%. Cause: the same error again, a harness that bypassed `pipeline.run`. The rule that follows is narrow and useful — **a configuration sweep must call the same entry point the product calls** — and it was *still* being broken, by the very script that produces the evidence behind the declaration. The clean-clone check found that third instance; the harness now asserts itself against `scanplan run` and exits non-zero if they diverge, because writing the rule down had already failed twice.

**Postscript: the gate later moved, and not because of any of this.** Applying the measured depth-bias correction took G-REPEAT-ROOMS to 5-vs-5 (**met**) while G-REPEAT-FOOTPRINT went 0.7% → 3.2% (**not met**). Exactly one of the two still passes and the correction flipped which — two gates that trade places under 18 mm of depth were never measuring independent properties. That is a side effect, not a third fix, and claiming it as one would repeat the original error of naming the right outcome and the wrong mechanism.

Full account: ``fix_loop.md``, \$6 for the post-mortem, \$9 for the reverted second attempt, \$10 for both of the above.

## 6. Validation

---

98 tests. The ones that matter assert **accuracy against geometry we constructed and therefore cannot be wrong about** — the fixture writes a real Stray Scanner export, so the actual loader is exercised,



not a mock. It also writes depth **18 mm short, exactly as the device does**, so the correction is exercised rather than bypassed.

Measured **through pipeline.run with the product's own defaults**, not through a configuration only the tests use:

| Quantity        | Truth                 | Measured              | Error   |
|-----------------|-----------------------|-----------------------|---------|
| Ceiling height  | 2.500 m               | 2.497 m               | −3.1 mm |
| Floor height    | 0.000 m               | +0.0015 m             | +1.5 mm |
| Floor area      | 12.000 m <sup>2</sup> | 11.994 m <sup>2</sup> | −0.05%  |
| Long dimension  | 4.000 m               | 3.950 m               | −50 mm  |
| Short dimension | 3.000 m               | 2.920 m               | −80 mm  |
| Perimeter       | 14.000 m              | 13.740 m              | −260 mm |
| Polygon         | rectangle             | 4 corners             | —       |

Area and the bounding extent disagree by about 4%, and that is by construction rather than by error: `floor_area_m2` is the floor actually covered by measurement, while the polygon is the regularised outline. The outline is the shape; the area is the measurement.

**The depth-bias correction is optimal on both quantities, and the ablation shows it.** Same fixture, same entry point, only the correction changed:

| depth correction                   | floor area    | ceiling height |
|------------------------------------|---------------|----------------|
| <b>0.018 m — measured, shipped</b> | <b>−0.05%</b> | <b>−3.1 mm</b> |
| 0.030 m — over by 12 mm            | +0.69%        | +17.3 mm       |
| 0.000 m — none                     | −1.44%        | −33.3 mm       |

Correcting by the measured value is better than both over-correcting and not correcting, on area and on ceiling height. That is the result a correct sensor model should produce, and it is worth stating because an earlier version of this document claimed something different.

**A correction to an earlier claim in this report.** It said the area figure had been flattered by two errors cancelling, and gave the honest value as about −3%. That −3% was itself a measurement artifact: the fixture under-sampled the floor, because its camera pitch cycled with period 3 and the pipeline's default stride is also 3. Fixing the fixture moved area to −0.05% and left the cancelling-errors explanation with nothing to explain. It is withdrawn rather than rewritten — the simpler reading, that the measured correction is simply right, is what the ablation above actually supports.

Damage detection is tested in both directions: a 40 × 30 cm patch lifted 25 mm is found with extent correct to 12 cm; a wardrobe 40 cm off the wall, and 4 mm of noise, are not reported. Three real undamaged flats produce zero regions.

Also pinned: intervals never collapse below the measured residual bias, log-scale intervals never go negative, unobserved values stay flagged, output is deterministic, no committed result carries a machine-specific path, the compliance matrix's gate table matches the benchmark, and the CLI never prints a traceback — in front of examiners a stated failure is worth more than a stack trace.

**A gate whose status moved with machine load.** A-RUNTIME had its measured seconds moved out of the result string so `gates.json` would reproduce — but the MET/NOT MET *status* still depended on wall-clock. Running the gates while another benchmark held the CPU pushed the worst capture from 44.6 s to 62.7 s and flipped the gate to NOT MET: a false failure, and by the same mechanism a quieter machine could produce a false pass. The gate now reads the 1-minute load average and reports **NOT MEASURED under contention**, with the load recorded in `timing.json`, because a timing taken on a busy machine does not answer the question the gate asks. Removing a number from a string is not the same as removing it from a decision.

**Reproduction.** `bench/clean_clone_check.sh` clones the repo, installs from scratch, runs the tests, regenerates the benchmarks and diffs committed against regenerated. It classifies every result file rather than counting files it never recomputed: an earlier version reported "8 identical" when seven of those had merely been copied by the clone and compared with themselves, which is a pass that cannot fail. It has caught three real defects — a bare `ModuleNotFoundError` on the default install, wall-clock seconds embedded in a gate *result*, and the fix-loop harness measuring an unshipped pipeline.

## 6b. Head to head against an independent implementation

An app cannot be handed our input. Another engineer's pipeline reading the same public capture format can — so [cozmo-scan](#), an independent submission to this brief, measures the **identical** synthetic capture of a room that is 4.00 × 3.00 m with a 2.50 m ceiling by construction. Truth is exact, not a tape reading.

**G-H2H-ENGINEER: beat or tie on 9 of 10 dimensions, 90%, MET.** Floor area −0.05% against +29.4%; the one loss is ceiling height on a capture modelling an *unbiased* sensor, where our 18 mm correction is unwarranted. Two captures are run on purpose — one modelling the device as measured, one an ideal sensor — because running only the one that suits our correction would be choosing the input without a single number changing.

This is **not** Part 3, which asks for a consumer scanning app. That is magicplan, and it is still PENDING for want of tape truth.

### Where the opponent is better, measured

Reporting only the synthetic result would be advertising, so the same benchmark runs both pipelines on the three supplied captures. Those carry no truth, so **nothing below is scored** and neither pipeline is right by default — but the divergence is the more useful question.

| Capture    | scanplan                      | cozmo-scan                            |
|------------|-------------------------------|---------------------------------------|
| c00a170fe1 | 2 rooms, 15.11 m <sup>2</sup> | <b>3 rooms</b> , 24.62 m <sup>2</sup> |
| 1a8384c3f6 | 5 rooms, 48.04 m <sup>2</sup> | <b>8 rooms</b> , 56.98 m <sup>2</sup> |
| c7d28f72c6 | 5 rooms, 49.65 m <sup>2</sup> | <b>9 rooms</b> , 56.07 m <sup>2</sup> |

**They split more rooms than we do on every capture.** That is the gate this project declared a fix for, predicted wrong, attempted a second time and reverted — and an independent implementation does it better. §7.5 states the under-splitting as a known failure; this is the measurement behind it, against another pipeline on the same data rather than against a reference number nobody can source.

**They also fit ceiling height per room where we fit one storey height per capture.** On c7d28f72c6 they report nine distinct heights from 2.29 to 3.10 m; we report 3.087 m for every room. Without truth neither is provably right, but a single 3.09 m across a whole flat is on the high side and a per-room model describes a real property more closely than ours does.

**And their frame selection is more robust than ours.** They choose keyframes adaptively by pose change; we use a fixed stride. That is precisely why [scanplan run](#) failed outright on a capture whose sweep aliased with our stride (§6) and their pipeline did not. We added a [--stride](#) escape hatch and a diagnostic rather than changing frame selection a day before the deadline, which is a mitigation and not a fix.

Two things stay in our favour and are worth stating plainly: **dimensional accuracy against exact truth**, where the gap is large and consistent across both generators and the convention is matched (their source specifies inside faces, as ours does); and the **measured depth bias**, which they carry as a 13 mm uncertainty rather than correcting.

## 6c. Wall-to-wall distance against laser truth

Ceiling height is the separation of two horizontal surfaces; a wall-to-wall distance is the same measurement turned ninety degrees. Once device and laser depth could be fused onto the same poses, this followed — and it measures the quantity a floor plan is actually made of.

**A-WALL-LIDAR: MET — 10 of 10 within max(2 cm, 1%), median 8.6 mm**, on the distances between the two walls *our device names*. That is a **stated narrowing** of the gate; the stricter reading, which also charges us for naming a different pair than the laser would, is **8 of 10, worst 225 mm** and is published in the same gate row. [docs/gates.md](#) has the definition; [docs/declined\\_changes.md](#) §2 records that this definition was rejected on first offer and why it is reported now. The split that motivated it:

|                                                 | distances | within gate | median  error |
|-------------------------------------------------|-----------|-------------|---------------|
| both clouds chose the <b>same</b> pair of walls | 8 of 10   | <b>8</b>    | <b>8.1 mm</b> |
| they chose <b>different</b> walls               | 2 of 10   | 0           | 194–225 mm    |

**On matched walls the sensor now agrees with the laser on every single one, at 8.1 mm median.** The two remaining failures are not sensor errors — they are our plane selection choosing a different pair of walls in one cloud than in the other. Each row publishes the coordinates it chose, so the two failure modes can be told apart; without them an error of that size is unreadable.

**What closed the gap: selection that knows orientation.** Density alone cannot tell a wall from a wardrobe side — along one horizontal axis a counter front, a sofa back and an actual wall all deposit a dense 1 cm column. Adding per-point normals, a facing test (the two walls must face *each other* across the room) and extent gates (a wall is long and tall; furniture fails one) took the worst row from **975 mm to 3.4 mm** and the median from 16.4 mm to 9.8 mm.

The approach is not ours: normals from the depth grid plus depth-edge rejection are from [cozmo-scan's fusion.py](#), credited in [bench/wall\\_normals.py](#). An earlier attempt at it was rejected for claiming independence it did not have, and for reaching MET via a second change that let the *device* choose which surfaces the laser then measured — relaxing what the gate penalises. [docs/declined\\_changes.md](#) §2 has that record; a test now guards against selection quietly becoming shared.

**The denominator fell from 12 to 10.** One walk has no two wall-sized surfaces facing each other on either axis, which is a failure to *find* a measurement rather than to make one accurately. It is recorded per row, because a silently dropped axis shrinks the denominator and reads as a measurement that happened to pass.

The denominator stays all twelve. Choosing the wrong walls is our error too, and scoring only the rows where our own selection happened to agree would convert a failure into a pass by discarding the failures.

**This does not measure the layout.** It measures the sensor and the fusion through our plane fitting on a room-scale distance. The wall lengths [scanplan](#) run emits still have no truth, because nobody has taped the supplied captures — the same limit A-WALL-LIDAR carried when it read NOT MEASURED, narrowed rather than removed.

**Honest comparison:** the independent submission measured this gate too and met it, 4 of 4 distances after depth correction. They measured a quarter as many distances and did better on them. Plane-pair stability is a real weakness on our side.

## 7. Known failure modes

---

- 1. No tape truth on the rooms we captured ourselves.** This is the largest remaining gap. It blocks the Part 3 head-to-head outright, where magicplan's own output is deliberately **not** used as truth because it is the opponent and scoring an opponent against itself is circular. What it does *not* block any more is absolute accuracy: three measurements rest on FARO laser depth — the **depth bias** per-pixel over 4.79 M pixels, **ceiling height** within 15 mm on 5 of 5 walks, and **wall-to-wall distance** (8 of 10, and 8/8 at 8.1 mm where our plane selection agrees). Public laser truth turned out to substitute for a tape on everything except our own two rooms.
- 2. The video tier is 27-60% from the LiDAR reference** on the same walks, and failed outright on the third capture with a stated **no floor found**. Its intervals are widened  $\times 11$  and do contain the reference 2/2, which is the correct response to an error of that size rather than a fix for it. Inferred depth is not measured depth, and the gap is the number this tier exists to report.
- 3. The photo tier under-reports room extent by 3-4 $\times$ . G-WALL-PHOTO: 0 of 6 within  $\pm 8\%$ , median error 75.5%.** Getting to the cause took three wrong answers, and the retractions matter more than the number:
  - **Published and WRONG:** "the inferred depth under-estimates scale". Measured per-pixel against the LiDAR depth on the same twelve stills, the inferred depth runs about **1.26 $\times$  LONG**. The depth model is not the problem, and this report said it was.
  - **Found and fixed, but not the binding constraint:** [\\_level\\_to\\_floor](#) assumed the camera was upright, requiring the floor normal within  $32^\circ$  of camera-y. On these frames camera-down is **92.8-93.8 $^\circ$  from world-down** — the phone was held turned — so the fit accepted a plane normal to camera-y, which is a **wall**, and levelled the room against it. A landscape photograph of a room is ordinary input, so the assumption was removed rather than documented: the floor is now identified by geometry alone, as the large plane with  $\geq 80\%$  of the room on one side of it. Walls fail that test; table tops fail it too.
  - **What the fix revealed:** it moved the gate from 76.8% to 75.5%. Essentially nothing. A real bug whose repair does not move the number is evidence about where the constraint actually is.
  - **Measured, and structural:** a single levelled view spans **2.2-2.9 m** in plan where the six-frame *posed* LiDAR reference spans **4.2-4.7 m**. With no poses the tier cannot merge views, so its box is bounded by what one viewpoint can reach. That is the limit [scanplan/ingest/photos.py](#) has always declared in its docstring; it is now quantified.

The gate stays NOT MET and the reference is the LiDAR tier on the same frames — a reference, not truth, since nobody has taped this property.

- 4. The photo tier does not stitch, by construction.** Two folders in, two disconnected groups out. With no camera poses nothing in the input says how rooms relate, and an L-shaped room returns as its bounding

box. The output carries an error-severity warning saying so rather than laying out a plausible arrangement.

5. **The two walks of one flat do not agree about what the rooms are, and G-REPEAT-ROOMS passing at "5 vs 5" is a count coincidence.** Both return five rooms. Paired by area rank they are 37%, 76%, 8%, 33% and 44% apart, and one walk keeps as a single room roughly what the other splits in two — while the total footprint agrees to 3.2%. The gate asks for the count, so it is met on its wording and the wording is a weak proxy; the gate table's result string now says "counts only" so it cannot be read as agreement. [bench/same\\_flat.py](#). This is also why per-wall **G-REPEAT is not measurable** here rather than merely unmeasured: even perfectly registered, a wall bounding a room in one walk runs through the middle of a room in the other, so there is no counterpart to match. Pairing by rank anyway and quoting a pass rate would rest on a correspondence that benchmark disproves.
6. **G-OPEN is measured and failed: 0 of 12 openings agree within 2 cm** between the two walks, the closest pair 4 cm apart. It needed no tape truth — only comparing one walk against the other, which had not been attempted. Separately and worse, **every opening we measure is 0.16-0.52 m wide where a doorway is 0.6-0.9 m**: the widths are not just irreproducible, they are too small. The clear-width measurement is under-reporting and that is unfixed.
7. **Rooms under-split** — 5 found on a capture where this project's own fix-loop declaration put the reference at 9. **That 9 has no source recorded anywhere in this repository.** It first appears in [fix\\_loop\\_declaration.md](#), which is committed-before-the-fix and therefore never edited, and it has been repeated since without anyone establishing where it came from. It is cited here because it is used *against* us, but an unsourced number is weak evidence in either direction and should not be relied on. What is measured: an independent implementation reports 9, 8 and 3 rooms on the three supplied captures where we report 5, 5 and 2 ([bench/results/head\\_to\\_head\\_engineer.json](#)). We under-split; that much is not in doubt. The safer failure: a merged pair still reports a correct combined area; an invented room does not.
8. **Damage class is shape-derived.** A stain that has not lifted the plaster is geometrically invisible. Every region says `class_source: "shape"` with confidence  $\leq 0.5$ , and detection has no verified true-positive rate because no real damaged capture was obtainable.
9. **Mirrors and glass are not detected.** Two geometric tests were built; neither separates a reflection from ordinary geometry — one fired on 19–30% of every capture, the other on 10–13%. The flag is **disabled deliberately** and every run warns that mirrors are undetected, which is worth more than a detector that fires everywhere. Wet/glossy floor and low light are detected and warned about.
10. **Room area is under-reported by design** — about –3% on known geometry. Coverage-based area cannot exceed what was seen, so occluded floor is missing rather than estimated.

## 8. What I would do next, in order

---

1. **Get tape truth onto two rooms.** It converts a whole column of *not measured* into numbers and unblocks the head-to-head. Highest value per hour by a wide margin — everything else on this list improves a number that is already known.
  2. **Stabilise the wall-plane selection**, which is now the clearest measured weakness. A-WALL-LIDAR agrees with the laser to **8.2 mm median** where both clouds choose the same pair of walls, and fails completely on the four rows where they do not. The sensor is fine; the densest-plane heuristic is not — it is a raw histogram argmax, so where two parallel surfaces 0.19-0.99 m apart have near-equal support the choice turns on a margin as small as **2.2%**, and the two clouds can land differently.
- Stated carefully, because an earlier version of this list over-claimed here: it is **not** established that fixing the heuristic would move the gate to most of 12. An "outermost supported plane" rule is stable on all the flipped rows, but it changes the measurand — on 41069046 it spans a doorway into the next room rather than a room width — and it was only identified by inspecting the rows that had already failed. Some of the ambiguity is in the scene (a full-height recess, a doorway on the axis), not only in the heuristic.
3. **Replace the global `door_max_m`** with a split that adapts to local room scale. Two parameter attempts have failed; \$5 is clear that this is not a parameter problem.
  4. **Pose estimation for the video tier**, so it works on a bare clip rather than needing a pose track. That is structure-from-motion, and it would need its own validation before any number it produced could be trusted.